

Examen2.pdf



sheeesh_



Programación y Diseño Orientado a Objetos



2º Grado en Ingeniería Informática



Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación
Universidad de Granada



MÁSTER EN

Inteligencia Artificial & Data Management

MADRID

Formamos
talento para un futuro
Sostenible

saber más



CAMBIA TU VIDA ESTE VERANO

4Geeks
academy

APRENDE A
PROGRAMAR

JAVASCRIPT

PYTHON - HTML

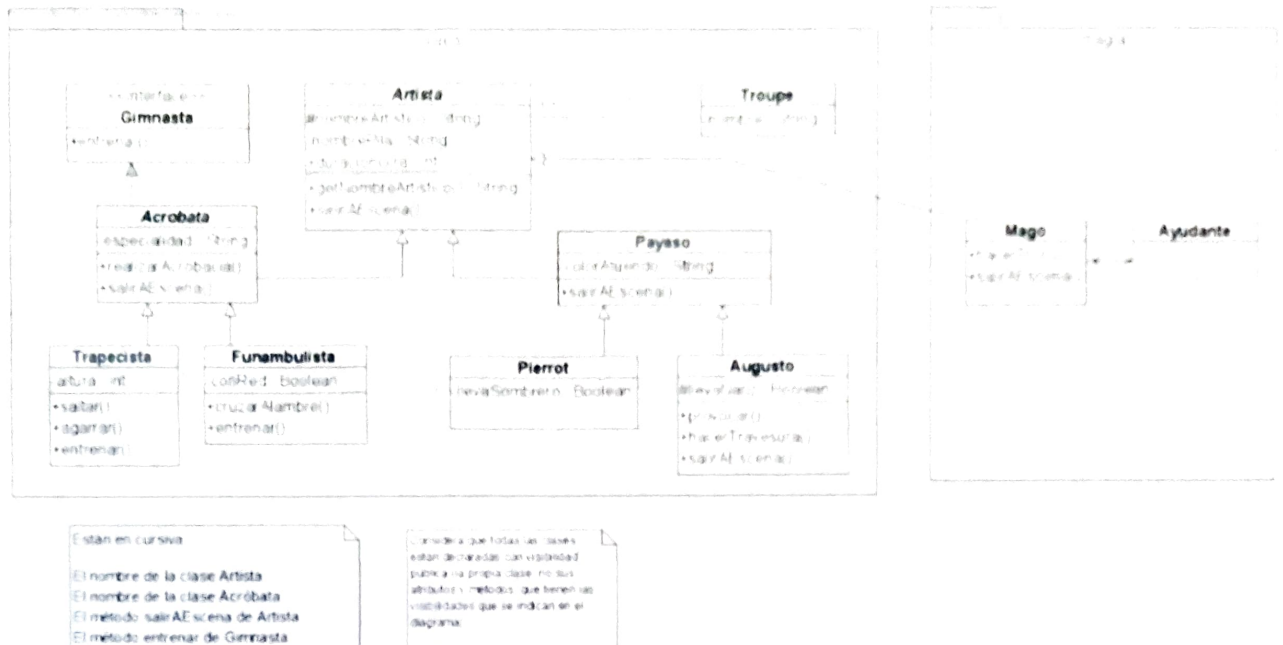
CSS - REACT

SQL - NODEJS

GIT - FLASK

GITHUB

BOOTSTRAP



- Supón que todas las clases del diagrama tienen un constructor que inicializa todos sus atributos utilizando valores que le llegan como argumento. Implementa en **Ruby** la cabecera de la clase Pierrot y su constructor.

```

package circo
public class Pierrot implements Payaso {
    private boolean llevaSombbrero
    public Pierrot (boolean llevaS , String colorAtuendo, String nArtistico, String nPila,
                    (int duracionGira) {
        super( colorAtuendo, nArtistico, nPila, duracionGira );
        this.llevaSombbrero = llevaS;
    }
}

```

- Implementa en **Ruby** los métodos salirAEscena de Payaso y de Augusto de modo que:

- en Payaso imprima por pantalla "¿Cómo están ustedes?"
- en Augusto haga lo que hace un Payaso y a continuación haga una travesura

```

class Payaso < Artista
...
def salirAEscena
    tmp = super
    tmp += "¿Cómo están ustedes?"
    puts tmp
end
...
end
class Augusto < Payaso
...
def salirAEscena
    tmp = super
    tmp += hacerTravesura()
    puts tmp
end
end

```

WUOLAH

3. Supón que tanto en Artista como en la subclase Trapecista tenemos los siguientes métodos de clase:

```
public static void setDuracionGira(int d){ duracionGira=d;}  
public static int getDuracionGira(){ return duracionGira;}
```

¿Qué se imprimiría con las siguientes líneas de código?

Asumiendo que `Artista = new Trapecista (...)` ;

```
Artista.setDuracionGira(2);  
System.out.println(Artista.getDuracionGira());
```

2

```
System.out.println(Trapecista.getDuracionGira());
```

2

```
Trapecista.setDuracionGira(4);  
System.out.println(Artista.getDuracionGira());
```

2

```
System.out.println(Trapecista.getDuracionGira());
```

4

¿Y si la subclase tuviese su propio atributo de la clase duracionGira inicializado a 4?

```
Artista.setDuracionGira(2);  
System.out.println(Artista.getDuracionGira());
```

2

```
System.out.println(Trapecista.getDuracionGira());
```

4

```
Trapecista.setDuracionGira(4);  
System.out.println(Artista.getDuracionGira());
```

2

```
System.out.println(Trapecista.getDuracionGira());
```

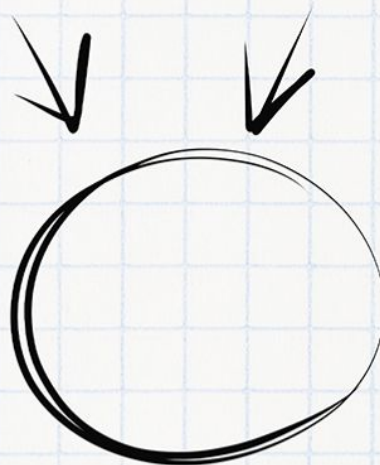
4

Imagínate aprobando el examen

Necesitas tiempo y concentración

Planes	 PLAN TURBO	 PLAN PRO	 PLAN PRO+
 Descargas sin publi al mes	10 	40 	80 
 Elimina el video entre descargas			
 Descarga carpetas			
 Descarga archivos grandes			
 Visualiza apuntes online sin publi			
 Elimina toda la publi web			
 Precios Anual ☐	0,99 € / mes	3,99 € / mes	7,99 € / mes

Ahora que puedes conseguirlo,
¿Qué nota vas a sacar?

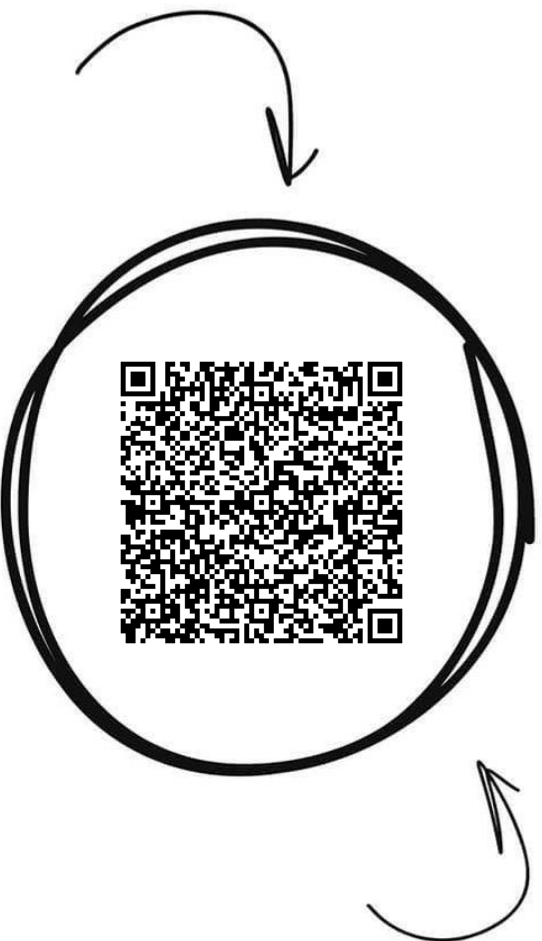


WUOLAH

Programación y Diseño Orient...



Comparte estos flyers en tu clase y consigue más dinero y recompensas



Banco de apuntes de la

MUOLAH

1

Imprime esta hoja

2

Recorta por la mitad

3

Coloca en un lugar visible para que tus compis puedan escanar y acceder a apuntes

4

Llévate dinero por cada descarga de los documentos descargados a través de tu QR



4. Supón que en la clase Pierrot existen los siguientes métodos adicionales donde compi es otro objeto Pierrot:

Ruby:

```
def presentacionCompi(compi)
  puts "Hoy está conmigo #{compi.nombreArtístico} vestido de
    #{compi.colorAtuendo}"
end
```

Java:

```
void autoPresentación(Pierrot compi){
    System.out.println("Me llamo " + nombreArtístico + " y voy vestido
de " + colorAtuendo);
}
```

Corrige los fallos en los métodos (si los hay). Si es necesario hacer algún cambio en alguna clase indícalo.

Ruby: - Hay que crear consultores para poder acceder a las variables nombreArtístico y color atuendo → attr_reader : nombreArtístico, colorAtuendo

Java: - Había que crear consultores para poder acceder a las variables usadas:
nombreArtístico tiene visibilidad de paquete
public String getColorAtuendo () { return colorAtuendo; }
- No hay que pasar ningún argumento a la función

Ejercicio 1: Ruby

```
module Circo
  class Pierrot < Payaso
    @llevaSombbrero
    def initialize ( llevaSombbrero, colorAtuendo, nombreArtístico, nombrePila )
      super( nombreArtístico, nombrePila, colorAtuendo )
      @llevaSombbrero = llevaSombbrero
    end
  end
end
```



**YA TIENES UN TÍTULO,
AHORA DA EL SALTO AL
MUNDO LABORAL.**

**POTENCIA TU PERFIL
CON LAS TECNOLOGÍAS
MÁS DEMANDADAS.**



Servicio de carreras
para que encuentres
curro en 180 días.

5. Indica en qué líneas no hay error, hay error de compilación o hay error de ejecución.

```
Artista x1 = new Acrobata();
```

Error de compilación → Acróbata también es abstracta

```
Gimnasta x2 = new Funambulista();
```

Sin errores

```
Payaso x3 = new Payaso();  
String s = x3.getNombreArtistico();
```

Sin errores

```
Acrobata x4 = new Trapecista();  
x4.agarrar();
```

Error de compilación → se puede resolver mediante casting : ((Trapecista) x4).agarrar();

```
Payaso pa2 = new Pierrot();  
((Augusto) pa2).salirAEscena();
```

Error de compilación → no se puede hacer casting entre hermanos (Pierrot y Augusto)

```
Artista pa3 = new Augusto();  
((Payaso) pa3).salirAEscena();
```

Sin errores

6. Supón que en nuestro sistema hay algunos magos mentalistas que no hacen trucos como cualquier otro mago, sino que leen la mente. Además, son capaces de hipnotizar, cosa que no cualquier mago puede hacer.

¿Cómo incluirías estas novedades en el sistema?

Java :

```
package magia;  
import java.circo.Artista;  
class Mentalista extends Mago {  
  
    Mentalista (String nombreArtístico,  
                String nombrePila) {  
        super(nombreArtístico, nombrePila);  
    }  
  
    @Override  
    public void hacerTruco () {  
        +mp = super();  
        + System.out.println("El mago te está  
        leyendo la mente");  
    }  
  
    @Override  
    public void hipnotizar () {  
        System.out.println("One ,two , Django!");  
    }  
}
```

Ruby :

```
module magia  
  require_relative ./circo/Artista.rb  
  
  class Mentalista < Mago  
    def initialize(nombreArtístico, nombrePila)  
      super(nombreArtístico, nombrePila)  
    end  
  
    def hacerTruco  
      +mp = super  
      +mp += "El mago te está leyendo la mente"  
      puts mp  
    end  
  
    def hipnotizar  
      +mp = "One ,two , Django!"  
      puts mp  
    end  
  end  
end
```

WUOLAH